

ChildProcess



Event-driven library for executing child processes with ReactPHP.

This library integrates Program Execution with the EventLoop. Child processes launched may be signaled and will emit an `exit` event upon termination. Additionally, process I/O streams (i.e. STDIN, STDOUT, STDERR) are exposed as Streams.

Table of contents

- Quickstart example
- Process
 - Stream Properties
 - Command
 - Termination
 - Custom pipes
 - Sigchild Compatibility
 - Windows Compatibility
- Install
- Tests
- License

Quickstart example

```
$process = new React\ChildProcess\Process('echo foo');
$process->start();

$process->stdout->on('data', function ($chunk) {
    echo $chunk;
});

$process->on('exit', function($exitCode, $termSignal) {
    echo 'Process exited with code ' . $exitCode . PHP_EOL;
});
```

See also the examples.

Process

Stream Properties

Once a process is started, its I/O streams will be constructed as instances of `React\Stream\ReadableStreamInterface` and `React\Stream\WritableStreamInterface`. Before `start()` is called, these properties are not set. Once a process terminates, the streams will become closed but not unset.

Following common Unix conventions, this library will start each child process with the three pipes matching the standard I/O streams as given below by default. You can use the named references for common use cases or access these as an array with all three pipes.

- `$stdin` or `$pipes[0]` is a `WritableStreamInterface`
- `$stdout` or `$pipes[1]` is a `ReadableStreamInterface`
- `$stderr` or `$pipes[2]` is a `ReadableStreamInterface`

Note that this default configuration may be overridden by explicitly passing custom pipes, in which case they may not be set or be assigned different values. In particular, note that Windows support is limited in that it doesn't support non-blocking `STDIO` pipes. The `$pipes` array will always contain references to all pipes as configured and the standard I/O references will always be set to reference the pipes matching the above conventions. See custom pipes for more details.

Because each of these implement the underlying `ReadableStreamInterface` or `WritableStreamInterface`, you can use any of their events and methods as usual:

```
$process = new Process($command);
$process->start();

$process->stdout->on('data', function ($chunk) {
    echo $chunk;
});

$process->stdout->on('end', function () {
    echo 'ended';
});

$process->stdout->on('error', function (Exception $e) {
    echo 'error: ' . $e->getMessage();
});

$process->stdout->on('close', function () {
    echo 'closed';
});

$process->stdin->write($data);
$process->stdin->end($data = null);
// ...
```

For more details, see the `ReadableStreamInterface` and `WritableStreamInterface`.

Command

The `Process` class allows you to pass any kind of command line string:

```
$process = new Process('echo test');  
$process->start();
```

The command line string usually consists of a whitespace-separated list with your main executable bin and any number of arguments. Special care should be taken to escape or quote any arguments, especially if you pass any user input along. Likewise, keep in mind that especially on Windows, it is rather common to have path names containing spaces and other special characters. If you want to run a binary like this, you will have to ensure this is quoted as a single argument using `escapeshellarg()` like this:

```
$bin = 'C:\\Program files (x86)\\PHP\\php.exe';  
$file = 'C:\\Users\\me\\Desktop\\Application\\main.php';  
  
$process = new Process(escapeshellarg($bin) . ' ' . escapeshellarg($file));  
$process->start();
```

By default, PHP will launch processes by wrapping the given command line string in a `sh` command on Unix, so that the first example will actually execute `sh -c echo test` under the hood on Unix. On Windows, it will not launch processes by wrapping them in a shell.

This is a very useful feature because it does not only allow you to pass single commands, but actually allows you to pass any kind of shell command line and launch multiple sub-commands using command chains (with `&&`, `||`, `;` and others) and allows you to redirect STDIO streams (with `2>&1` and family). This can be used to pass complete command lines and receive the resulting STDIO streams from the wrapping shell command like this:

```
$process = new Process('echo run && demo || echo failed');  
$process->start();
```

Note that Windows support is limited in that it doesn't support STDIO streams at all and also that processes will not be run in a wrapping shell by default. If you want to run a shell built-in function such as `echo hello` or `sleep 10`, you may have to prefix your command line with an explicit shell like `cmd /c echo hello`.

In other words, the underlying shell is responsible for managing this command line and launching the individual sub-commands and connecting their STDIO streams as appropriate. This implies that the `Process` class will only receive the resulting STDIO streams from the wrapping shell, which will thus contain the complete input/output with no way to discern the input/output of single sub-commands.

If you want to discern the output of single sub-commands, you may want to

implement some higher-level protocol logic, such as printing an explicit boundary between each sub-command like this:

```
$process = new Process('cat first && echo --- && cat second');
$process->start();
```

As an alternative, considering launching one process at a time and listening on its `exit` event to conditionally start the next process in the chain. This will give you an opportunity to configure the subsequent process I/O streams:

```
$first = new Process('cat first');
$first->start();

$first->on('exit', function () {
    $second = new Process('cat second');
    $second->start();
});
```

Keep in mind that PHP uses the shell wrapper for ALL command lines on Unix. While this may seem reasonable for more complex command lines, this actually also applies to running the most simple single command:

```
$process = new Process('yes');
$process->start();
```

This will actually spawn a command hierarchy similar to this on Unix:

```
5480 ... \_ php example.php
5481 ...   \_ sh -c yes
5482 ...     \_ yes
```

This means that trying to get the underlying process PID or sending signals will actually target the wrapping shell, which may not be the desired result in many cases.

If you do not want this wrapping shell process to show up, you can simply prepend the command string with `exec` on Unix platforms, which will cause the wrapping shell process to be replaced by our process:

```
$process = new Process('exec yes');
$process->start();
```

This will show a resulting command hierarchy similar to this:

```
5480 ... \_ php example.php
5481 ...   \_ yes
```

This means that trying to get the underlying process PID and sending signals will now target the actual command as expected.

Note that in this case, the command line will not be run in a wrapping shell. This implies that when using `exec`, there's no way to pass command lines such as those containing command chains or redirected STDIO streams.

As a rule of thumb, most commands will likely run just fine with the wrapping shell. If you pass a complete command line (or are unsure), you **SHOULD** most likely keep the wrapping shell. If you're running on Unix and you want to pass an individual command only, you **MAY** want to consider prepending the command string with **exec** to avoid the wrapping shell.

Termination

The **exit** event will be emitted whenever the process is no longer running. Event listeners will receive the exit code and termination signal as two arguments:

```
$process = new Process('sleep 10');
$process->start();

$process->on('exit', function ($code, $term) {
    if ($term === null) {
        echo 'exit with code ' . $code . PHP_EOL;
    } else {
        echo 'terminated with signal ' . $term . PHP_EOL;
    }
});
```

Note that **\$code** is **null** if the process has terminated, but the exit code could not be determined (for example **sigchild** compatibility was disabled). Similarly, **\$term** is **null** unless the process has terminated in response to an uncaught signal sent to it. This is not a limitation of this project, but actual how exit codes and signals are exposed on POSIX systems, for more details see also [here](#).

It's also worth noting that process termination depends on all file descriptors being closed beforehand. This means that all process pipes will emit a **close** event before the **exit** event and that no more **data** events will arrive after the **exit** event. Accordingly, if either of these pipes is in a paused state (**pause()** method or internally due to a **pipe()** call), this detection may not trigger.

The **terminate(?int \$signal = null): bool** method can be used to send the process a signal (**SIGTERM** by default). Depending on which signal you send to the process and whether it has a signal handler registered, this can be used to either merely signal a process or even forcefully terminate it.

```
$process->terminate(SIGUSR1);
```

Keep the above section in mind if you want to forcefully terminate a process. If your process spawn sub-processes or implicitly uses the wrapping shell mentioned above, its file descriptors may be inherited to child processes and terminating the main process may not necessarily terminate the whole process tree. It is highly suggested that you explicitly **close()** all process pipes accordingly when terminating a process:

```
$process = new Process('sleep 10');
```

```

$process->start();

Loop::addTimer(2.0, function () use ($process) {
    foreach ($process->pipes as $pipe) {
        $pipe->close();
    }
    $process->terminate();
});

```

For many simple programs these seemingly complicated steps can also be avoided by prefixing the command line with `exec` to avoid the wrapping shell and its inherited process pipes as mentioned above.

```

$process = new Process('exec sleep 10');
$process->start();

Loop::addTimer(2.0, function () use ($process) {
    $process->terminate();
});

```

Many command line programs also wait for data on `STDIN` and terminate cleanly when this pipe is closed. For example, the following can be used to “soft-close” a `cat` process:

```

$process = new Process('cat');
$process->start();

Loop::addTimer(2.0, function () use ($process) {
    $process->stdin->end();
});

```

While process pipes and termination may seem confusing to newcomers, the above properties actually allow some fine grained control over process termination, such as first trying a soft-close and then applying a force-close after a timeout.

Custom pipes

Following common Unix conventions, this library will start each child process with the three pipes matching the standard I/O streams by default. For more advanced use cases it may be useful to pass in custom pipes, such as explicitly passing additional file descriptors (FDs) or overriding default process pipes.

Note that passing custom pipes is considered advanced usage and requires a more in-depth understanding of Unix file descriptors and how they are inherited to child processes and shared in multi-processing applications.

If you do not want to use the default standard I/O pipes, you can explicitly pass an array containing the file descriptor specification to the constructor like this:

```

$fds = array(
    // standard I/O pipes for stdin/stdout/stderr
    0 => array('pipe', 'r'),
    1 => array('pipe', 'w'),
    2 => array('pipe', 'w'),

    // example FDs for files or open resources
    4 => array('file', '/dev/null', 'r'),
    6 => fopen('log.txt', 'a'),
    8 => STDERR,

    // example FDs for sockets
    10 => fsockopen('localhost', 8080),
    12 => stream_socket_server('tcp://0.0.0.0:4711')
);

$process = new Process($cmd, null, null, $fds);
$process->start();

```

Unless your use case has special requirements that demand otherwise, you're highly recommended to (at least) pass in the standard I/O pipes as given above. The file descriptor specification accepts arguments in the exact same format as the underlying `proc_open()` function.

Once the process is started, the `$pipes` array will always contain references to all pipes as configured and the standard I/O references will always be set to reference the pipes matching common Unix conventions. This library supports any number of pipes and additional file descriptors, but many common applications being run as a child process will expect that the parent process properly assigns these file descriptors.

Sigchild Compatibility

Internally, this project uses a work-around to improve compatibility when PHP has been compiled with the `--enable-sigchild` option. This should not affect most installations as this configure option is not used by default and many distributions (such as Debian and Ubuntu) are known to not use this by default. Some installations that use Oracle OCI8 may use this configure option to circumvent `defunct` processes.

When PHP has been compiled with the `--enable-sigchild` option, a child process' exit code cannot be reliably determined via `proc_close()` or `proc_get_status()`. To work around this, we execute the child process with an additional pipe and use that to retrieve its exit code.

This work-around incurs some overhead, so we only trigger this when necessary and when we detect that PHP has been compiled with the `--enable-sigchild` option. Because PHP does not provide a way to reliably detect this option, we

try to inspect output of PHP's configure options from the `phpinfo()` function.

The static `setSigchildEnabled(bool $sigchild): void` method can be used to explicitly enable or disable this behavior like this:

```
// advanced: not recommended by default
Process::setSigchildEnabled(true);
```

Note that all processes instantiated after this method call will be affected. If this work-around is disabled on an affected PHP installation, the `exit` event may receive `null` instead of the actual exit code as described above. Similarly, some distributions are known to omit the configure options from `phpinfo()`, so automatic detection may fail to enable this work-around in some cases. You may then enable this explicitly as given above.

Note: The original functionality was taken from Symfony's Process component.

Windows Compatibility

Due to platform constraints, this library provides only limited support for spawning child processes on Windows. In particular, PHP does not allow accessing standard I/O pipes on Windows without blocking. As such, this project will not allow constructing a child process with the default process pipes and will instead throw a `LogicException` on Windows by default:

```
// throws LogicException on Windows
$process = new Process('ping example.com');
$process->start();
```

There are a number of alternatives and workarounds as detailed below if you want to run a child process on Windows, each with its own set of pros and cons:

- As of PHP 8, you can start the child process with `socket` pair descriptors in place of normal standard I/O pipes like this:

```
$process = new Process(
    'ping example.com',
    null,
    null,
    [
        ['socket'],
        ['socket'],
        ['socket']
    ]
);
$process->start();

$process->stdout->on('data', function ($chunk) {
    echo $chunk;
});
```


These `socket` pairs support non-blocking process I/O on any platform, including Windows. However, not all programs accept stdio sockets.

- This package does work on Windows Subsystem for Linux (or WSL) without issues. When you are in control over how your application is deployed, we recommend installing WSL when you want to run this package on Windows.
- If you only care about the exit code of a child process to check if its execution was successful, you can use custom pipes to omit any standard I/O pipes like this:

```
$process = new Process('ping example.com', null, null, array());
$process->start();

$process->on('exit', function ($exitcode) {
    echo 'exit with ' . $exitcode . PHP_EOL;
});
```

Similarly, this is also useful if your child process communicates over sockets with remote servers or even your parent process using the `Socket` component. This is usually considered the best alternative if you have control over how your child process communicates with the parent process.

- If you only care about command output after the child process has been executed, you can use custom pipes to configure file handles to be passed to the child process instead of pipes like this:

```
$process = new Process('ping example.com', null, null, array(
    array('file', 'nul', 'r'),
    $stdout = tmpfile(),
    array('file', 'nul', 'w')
));
$process->start();

$process->on('exit', function ($exitcode) use ($stdout) {
    echo 'exit with ' . $exitcode . PHP_EOL;

    // rewind to start and then read full file (demo only, this is blocking).
    // reading from shared file is only safe if you have some synchronization in place
    // or after the child process has terminated.
    rewind($stdout);
    echo stream_get_contents($stdout);
    fclose($stdout);
});
```

Note that this example uses `tmpfile()/fopen()` for illustration purposes only. This should not be used in a truly async program because the filesystem is inherently blocking and each call could potentially take several

seconds. See also the Filesystem component as an alternative.

- If you want to access command output as it happens in a streaming fashion, you can use redirection to spawn an additional process to forward your standard I/O streams to a socket and use custom pipes to omit any actual standard I/O pipes like this:

```
$server = new React\Socket\Server('127.0.0.1:0');
$server->on('connection', function (React\Socket\ConnectionInterface $connection) {
    $connection->on('data', function ($chunk) {
        echo $chunk;
    });
});

$command = 'ping example.com | foobar ' . escapeshellarg($server->getAddress());
$process = new Process($command, null, null, array());
$process->start();

$process->on('exit', function ($exitcode) use ($server) {
    $server->close();
    echo 'exit with ' . $exitcode . PHP_EOL;
});
```

Note how this will spawn another fictional `foobar` helper program to consume the standard output from the actual child process. This is in fact similar to the above recommendation of using socket connections in the child process, but in this case does not require modification of the actual child process.

In this example, the fictional `foobar` helper program can be implemented by simply consuming all data from standard input and forwarding it to a socket connection like this:

```
$socket = stream_socket_client($argv[1]);
do {
    fwrite($socket, $data = fread(STDIN, 8192));
} while (isset($data[0]));
```

Accordingly, this example can also be run with plain PHP without having to rely on any external helper program like this:

```
$code = '$s=stream_socket_client($argv[1]);do{fwrite($s,$d=fread(STDIN, 8192));}while(1)';
$command = 'ping example.com | php -r ' . escapeshellarg($code) . ' ' . escapeshellarg($code);
$process = new Process($command, null, null, array());
$process->start();
```

See also example #23.

Note that this is for illustration purposes only and you may want to implement some proper error checks and/or socket verification in actual

production use if you do not want to risk other processes connecting to the server socket. In this case, we suggest looking at the excellent `createprocess-windows`.

Additionally, note that the command given to the `Process` will be passed to the underlying Windows-API (`CreateProcess`) as-is and the process will not be launched in a wrapping shell by default. In particular, this means that shell built-in functions such as `echo hello` or `sleep 10` may have to be prefixed with an explicit shell command like this:

```
$process = new Process('cmd /c echo hello', null, null, $pipes);  
$process->start();
```

Install

The recommended way to install this library is through Composer. New to Composer?

This will install the latest supported version:

```
composer require react/child-process:^0.6.6
```

See also the CHANGELOG for details about version upgrades.

This project aims to run on any platform and thus does not require any PHP extensions and supports running on legacy PHP 5.3 through current PHP 8+ and HHVM. It's *highly recommended to use the latest supported PHP version* for this project.

See above note for limited Windows Compatibility.

Tests

To run the test suite, you first need to clone this repo and then install all dependencies through Composer:

```
composer install
```

To run the test suite, go to the project root and run:

```
vendor/bin/phpunit
```

License

MIT, see LICENSE file.